



Оркестрация вычислений в биоинформатике

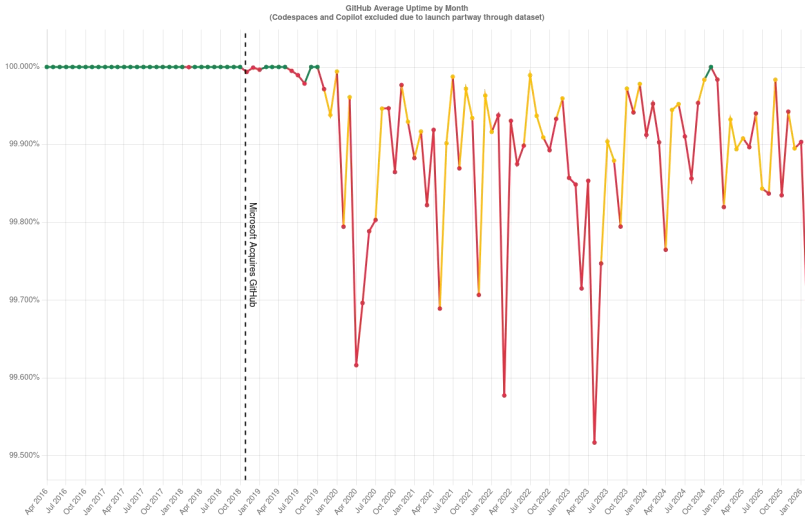
Лекция 10. Оркестрация. Apache Airflow.
DAG. Операторы. Jinja. Ветвление.

Оркестрация

Оркестрация может быть сервисная, а не DE: **k8s, Docker Swarm, podman.**

Созданы для бесперебойной работы системы (5 nines и др.).

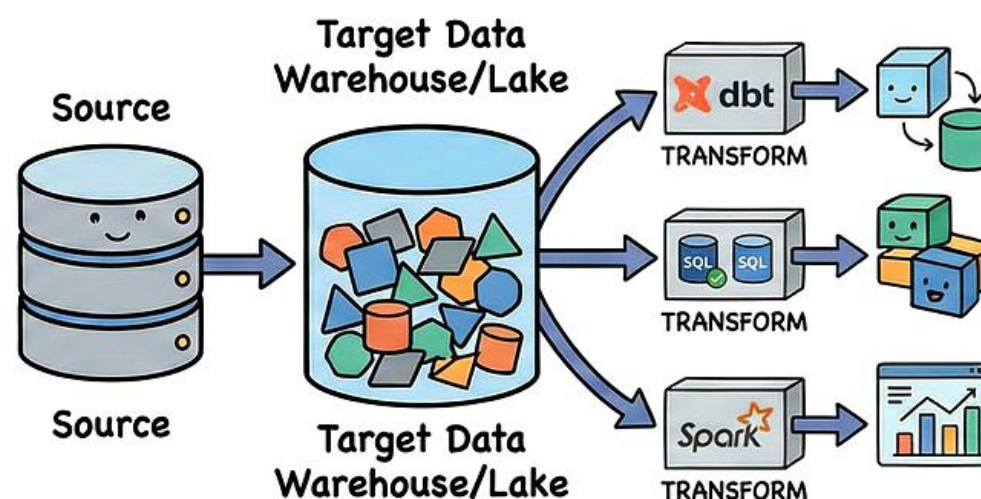
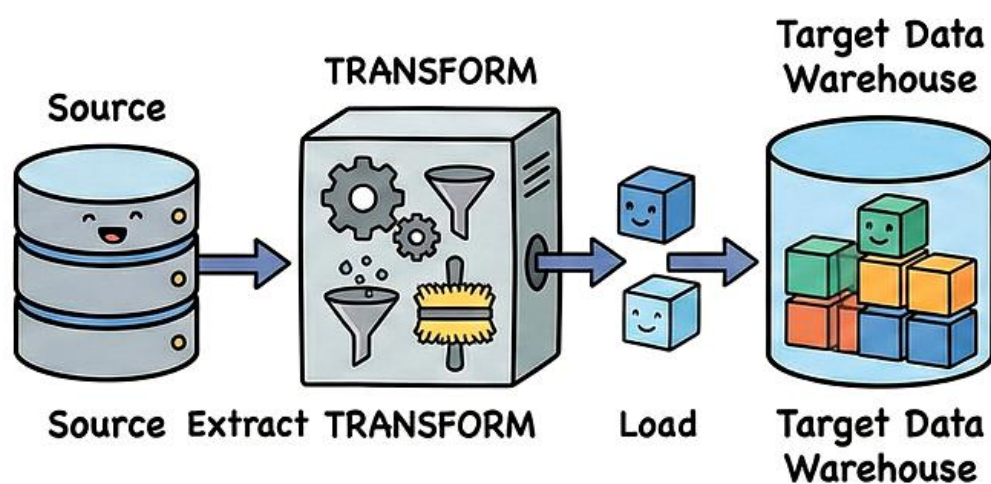
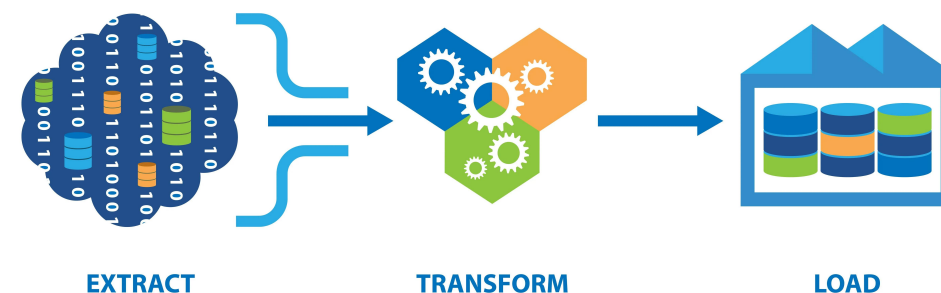
Availability	Downtime per Year (365 days & 24 hour cycle)
99.9999%	32 Seconds
99.999%	5 minutes, 15 seconds
99.99%	52 minutes, 36 seconds
99.95%	4 hours, 23 minutes
99.9%	8 Hours, 46 minutes
99.5%	1 day, 19 hours, 48 minutes
99%	3 days, 15 hours, 40 minutes



Оркестрация и ETL

Задача биоинформатика скорее другая:
вытащить данные, обработать, положить в
удобном виде для анализа (опционально -
обучить модель).

Это называется ETL:
Extract, Transform, Load
(иногда T/L меняют местами для разных систем)



Оркестрация

Не хочется сидеть над компьютером в ожидании

Писать bash мастер-скрипты полезно, но не поможет планировать вычисления, форматировать ошибки единым способом для разных кейсов.

Выход:
системы-оркестраторы

На самом деле,
**bash + cron почти
достаточно:**



А если между пайплайнами надо передавать данные?

Оркестрация

Не хочется сидеть над компьютером в ожидании

Писать bash мастер-скрипты полезно, но не поможет планировать вычисления, форматировать ошибки единым способом для разных кейсов.

Выход:
системы-оркестраторы

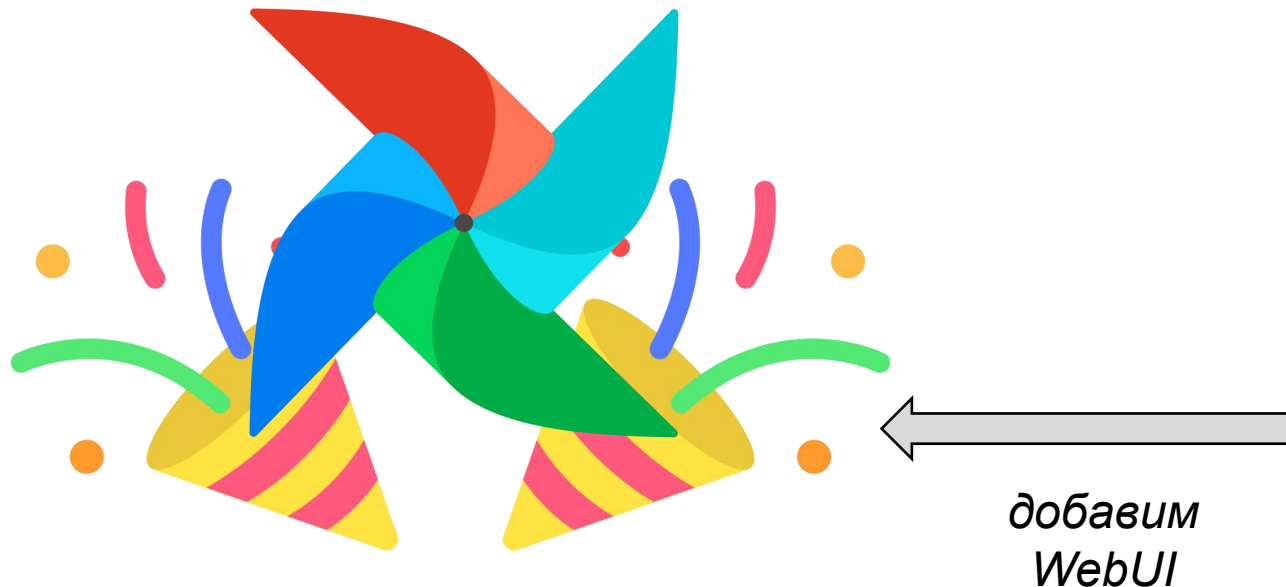
На самом деле,
**bash + cron почти
достаточно:**



А если между пайплайнами надо передавать данные?

**Пусть пишут в базу метаданных!
(обычная БД)**

Оркестрация



У нас есть всё, чтобы
построить Airflow или
любой другой оркестратор,
работающий верно.

**На самом деле,
bash + cron почти
достаточно:**



А если между пайплайнами надо
передавать данные?

**Пусть пишут в базу метаданных!
(обычная БД)**

Honorable mentions

Другие популярные системы, которые мы не посмотрим:



 nextflow



COMMON
WORKFLOW
LANGUAGE



{wdl}

Uti9t

Почему Airflow?

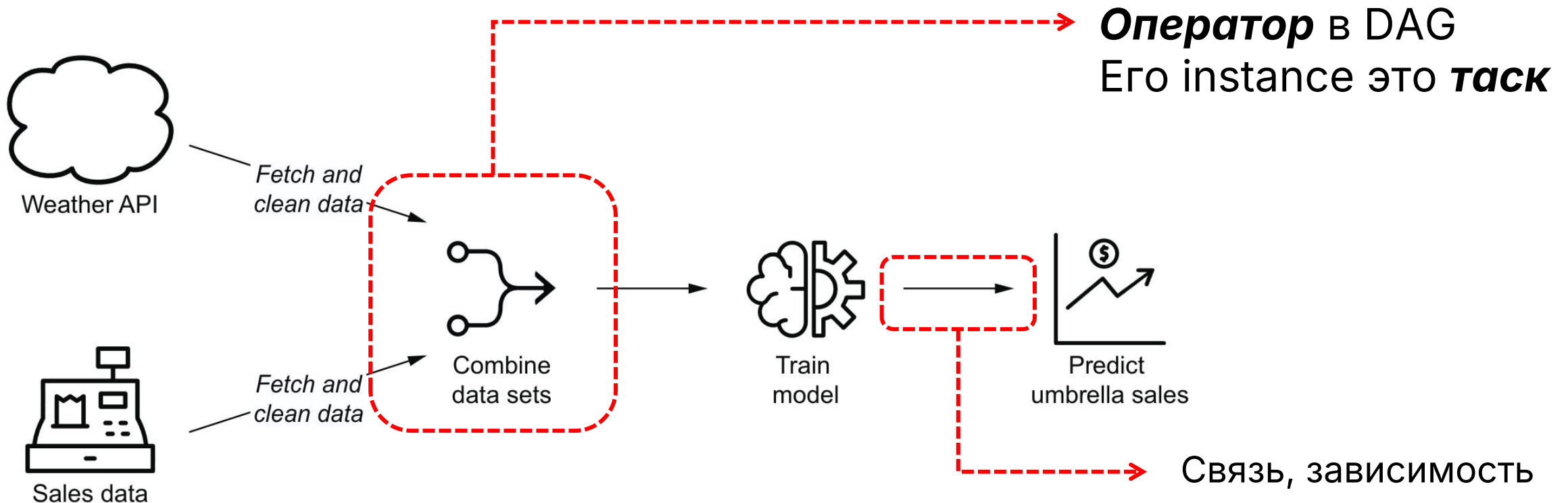
- [+] Pythonic (без усложнений Prefect)
- [+] Возможности мониторинга и ML(Ops)
- [+] Production-ready
- [+] Интеграция с HPC (но можно гонять контейнеры!)

Основная реальная альтернатива: `snakemake` и `nextflow`.

Устройство Airflow

Основной элемент -- DAG = пайплайн.

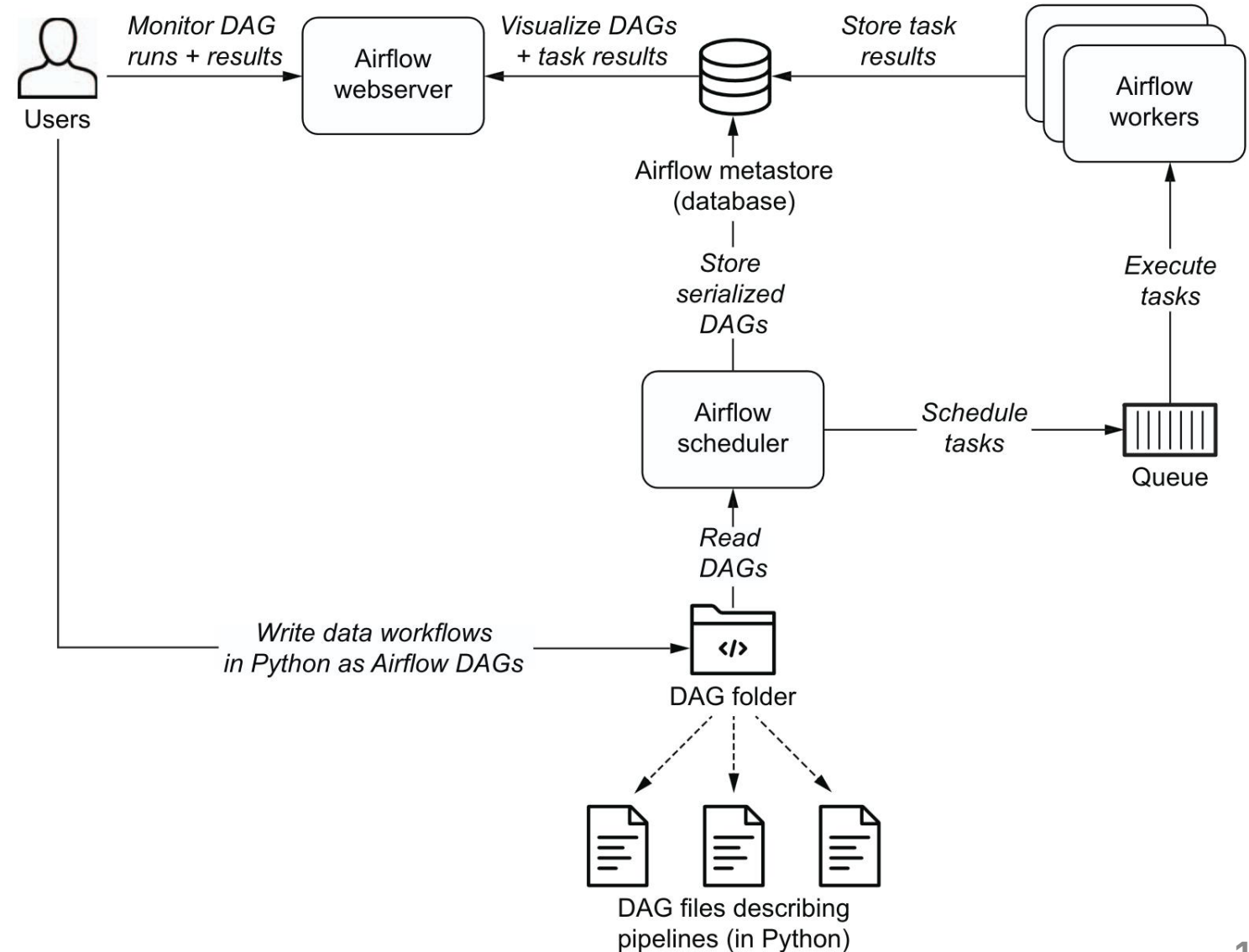
Пришло из того, что airflow -- часть экосистемы Apache.



Устройство Airflow

Компоненты:

- Scheduler
- Worker(s)
- Webserver
- БД метаданных
- (брокер)



Инициализация

Для работы airflow надо инициализировать БД метаданных и юзера в ней

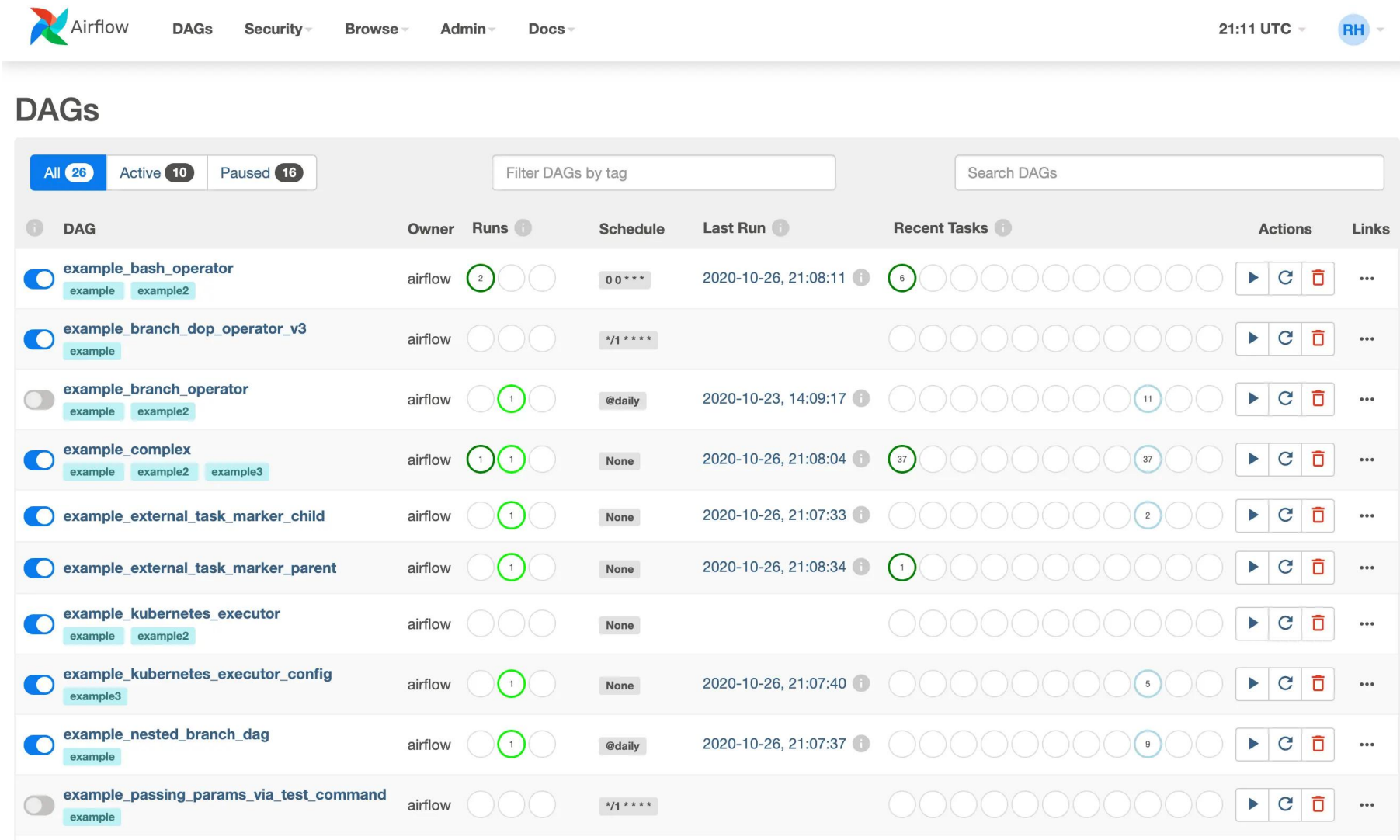
```
export AIRFLOW_HOME=~/.airflow
airflow db init

airflow users create --username admin \
  --firstname Demo --lastname User \
  --role Admin --email fake@mail.com \
  --password qwerty12345

airflow webserver --port 8080
airflow scheduler
```

Webserver

Основной инструмент взаимодействия и отладки.



The screenshot shows the Apache Airflow web interface. At the top, there's a navigation bar with the Airflow logo, links for DAGs, Security, Browse, Admin, and Docs, the current time (21:11 UTC), and a user profile icon (RH). Below the navigation bar, the 'DAGs' section is active. It features a filter bar with 'All 26', 'Active 10', and 'Paused 16' buttons, a 'Filter DAGs by tag' input, and a 'Search DAGs' input. The main table lists various DAGs with columns for DAG name, Owner, Runs, Schedule, Last Run, Recent Tasks, Actions, and Links. Each DAG row includes a toggle switch, a status icon (blue for active, grey for paused), a tag list, the owner name, a runs counter with a green circle around the number, a schedule field, a last run timestamp with an info icon, a recent tasks counter with a green circle around the number, a series of task status icons, and action buttons (play, refresh, delete) and a links menu.

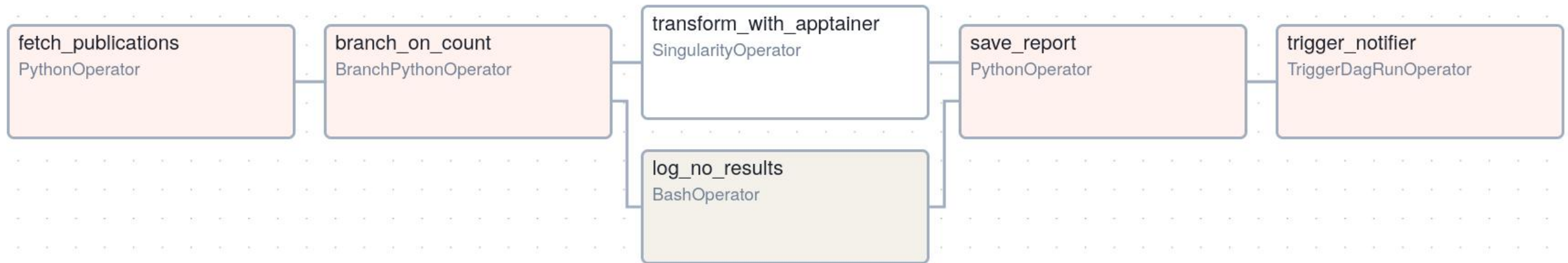
DAG	Owner	Runs	Schedule	Last Run	Recent Tasks	Actions	Links
☒ example_bash_operator example example2	airflow	2	0 0 ***	2020-10-26, 21:08:11	6		...
☒ example_branch_dop_operator_v3 example	airflow		* / 1 * * * *				...
☐ example_branch_operator example example2	airflow	1	@daily	2020-10-23, 14:09:17	11		...
☒ example_complex example example2 example3	airflow	1 1	None	2020-10-26, 21:08:04	37		...
☒ example_external_task_marker_child	airflow	1	None	2020-10-26, 21:07:33	2		...
☒ example_external_task_marker_parent	airflow	1	None	2020-10-26, 21:08:34	1		...
☒ example_kubernetes_executor example example2	airflow		None				...
☒ example_kubernetes_executor_config example3	airflow	1	None	2020-10-26, 21:07:40	5		...
☒ example_nested_branch_dag example	airflow	1	@daily	2020-10-26, 21:07:37	9		...
☐ example_passing_params_via_test_command example	airflow		* / 1 * * * *				...

Вход по кредам,
указанным при
создании юзера
в airflow при
инициализации

Создаём DAG

Какую задачу решим?

Пусть DAG принимает на вход название гена, проверяет наличие статей в NCBI по нему, и возвращает JSON, если они есть.

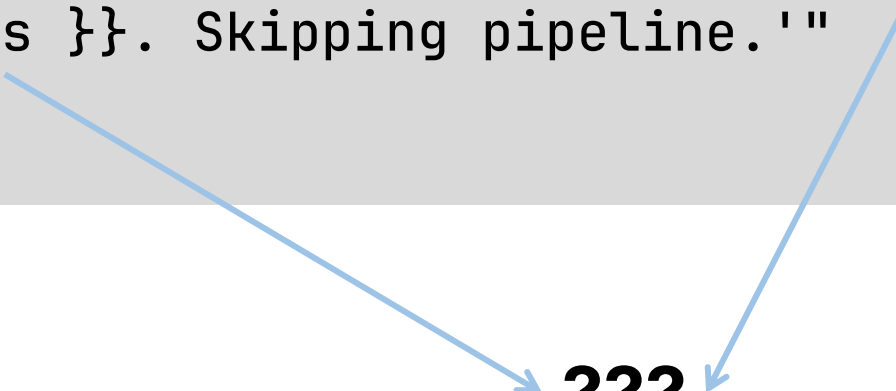


Операторы в airflow

Объекты, в которых прописана основная работа.
BashOperator, PythonOperator, SingularityOperator, ...
Когда нужно обработать, загрузить, сохранить

```
log_no_results = BashOperator(  
    task_id="log_no_results",  
    bash_command=(  
        "echo 'No results for gene={{ params.gene }}"  
        "on date={{ ds }}. Skipping pipeline.'"  
    ),  
)
```

???



Шаблонизация

Значения переменных в оркестрации можно шаблонизировать. А именно:

- аргументы операторов



Jinja

```
get_data = BashOperator(  
    task_id="get_data",  
    bash_command=(  
        "curl -o /tmp/wikipageviews.gz "  
        "https://dumps.wikimedia.org/other/pageviews/"  
        "{{ execution_date.year }}/"  
        "{{ execution_date.year }}-"  
        "{{ '{:02}'.format(execution_date.month) }}/"  
        "pageviews-{{ execution_date.year }}"  
        "{{ '{:02}'.format(execution_date.month) }}"  
        "{{ '{:02}'.format(execution_date.day) }}-"  
        "{{ '{:02}'.format(execution_date.hour) }}0000.gz"  
    ),  
    dag=dag,  
)
```


Шаблонизация

Значения переменных в оркестрации можно шаблонизировать. А именно:

- аргументы операторов
- **ID ДАГ, таска, оператора, их время...**



Jinja

```
def _print_context(**kwargs):  
    print(kwargs)
```

```
print_context = PythonOperator(  
    task_id="print_context",  
    python_callable=_print_context,  
    dag=dag,  
)
```



```
{  
    'dag': <DAG: print_context>,  
    'ds': '2019-07-04',  
    'next_ds': '2019-07-04',  
    'next_ds_nodash': '20190704',  
    'prev_ds': '2019-07-03',  
    'prev_ds_nodash': '20190703',  
    ...  
}
```

Шаблонизация

Значения переменных в оркестрации можно шаблонизировать. А именно:

- аргументы операторов
- ~~- ID ДАГ, задача, оператора, их время~~



Jinja

ПЕРЕМЕННЫЕ КОНТЕКСТА!

```
def _print_context(**kwargs):  
    print(kwargs)
```

```
print_context = PythonOperator(  
    task_id="print_context",  
    python_callable=_print_context,  
    dag=dag,  
)
```



```
{  
    'dag': <DAG: print_context>,  
    'ds': '2019-07-04',  
    'next_ds': '2019-07-04',  
    'next_ds_nodash': '20190704',  
    'prev_ds': '2019-07-03',  
    'prev_ds_nodash': '20190703',  
    ...  
}
```

Операторы в airflow

Объекты, в которых прописана основная работа

```
fetch_publications = PythonOperator(
    task_id="fetch_publications",
    python_callable=_fetch_publications,
    op_kwargs={
        "gene": "{{ params.gene }}",
        "max_results": "{{ params.max_results }}",
        "ds": "{{ ds }}"
    },
)
```



```
def _fetch_publications(gene: str, max_results: int, ds: str, **context) -> int:
    """
    Call NCBI search to find PubMed IDs for *gene* published in the
    year of the DAG execution date. Pushes a result dict to XCom and
    writes raw JSON to DATA_DIR.

    Returns the total hit count (used as the task return value, also
    automatically pushed to XCom under key 'return_value').
    """
    import requests # provider import inside callable to keep module-level clean

    os.makedirs(DATA_DIR, exist_ok=True)

    year = ds[0:4]
    params = {
        "db": "pubmed",
        "term": f"{gene}[Date] AND ({year})[PUB]",
        "retmax": 1000,
        "retmode": "json",
        "usehistory": "y",
    }
    resp = requests.get(f"https://eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi", params=params, timeout=30)
    resp.raise_for_status()

    esresult = resp.json()["esearchresult"]
    payload = {
        "gene": gene,
        "execution_date": ds,
        "count": int(esresult["count"]),
        "ids": esresult["idlist"],
        "webenv": esresult.get("webenv", ""),
        "query_key": esresult.get("querykey", ""),
    }

    raw_path = os.path.join(DATA_DIR, f"raw_{gene}_{ds}.json")
    with open(raw_path, "w") as fh:
        json.dump(payload, fh, indent=2)

    ti = context["ti"]
    ti.xcom_push(key="ncbi_payload", value=payload)
    ti.xcom_push(key="raw_path", value=raw_path)

    print(f"[fetch] gene={gene} year={year} hits={payload['count']} ids={payload['ids']}")
    return payload["count"]
```

какая-то логика
элементарного
шага ETL

Операторы в airflow

Объекты, в которых прописана основная работа

```
save_report = PythonOperator(  
    task_id="save_report",  
    python_callable=_save_report,  
    op_kwargs={  
        "gene": "{{ params.gene }}",  
        "ds": "{{ ds }}",  
    },  
    trigger_rule=  
        TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,  
)
```

У оператора могут быть разные правила старта (триггеры):

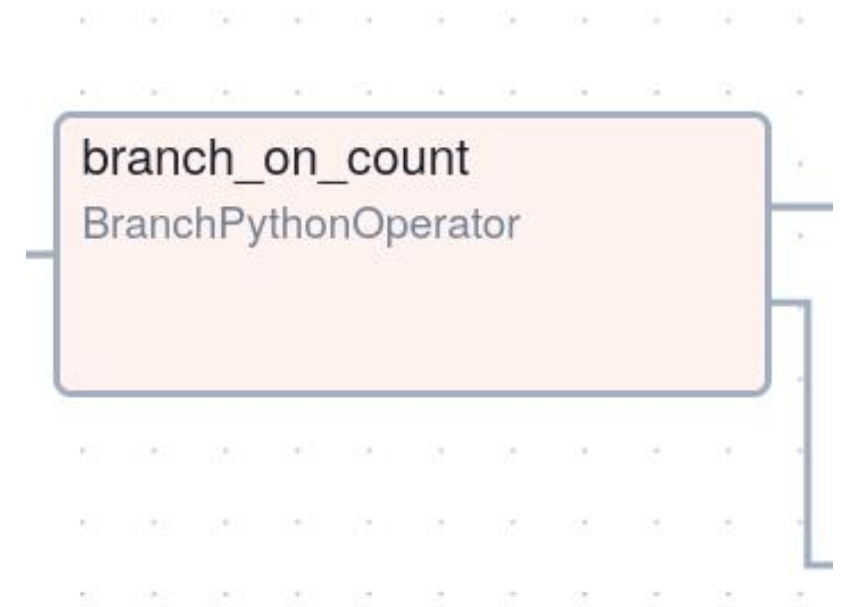
{one, none, all}_{success, fail, done, skipped}, always, ...

Операторы в airflow

Могут реализовывать базовую логику ветвления.

```
branch_on_count = BranchPythonOperator(  
    task_id="branch_on_count",  
    python_callable=_branch_on_count,  
)
```

Возвращать вызываемая
функция будет task ID
следующего оператора

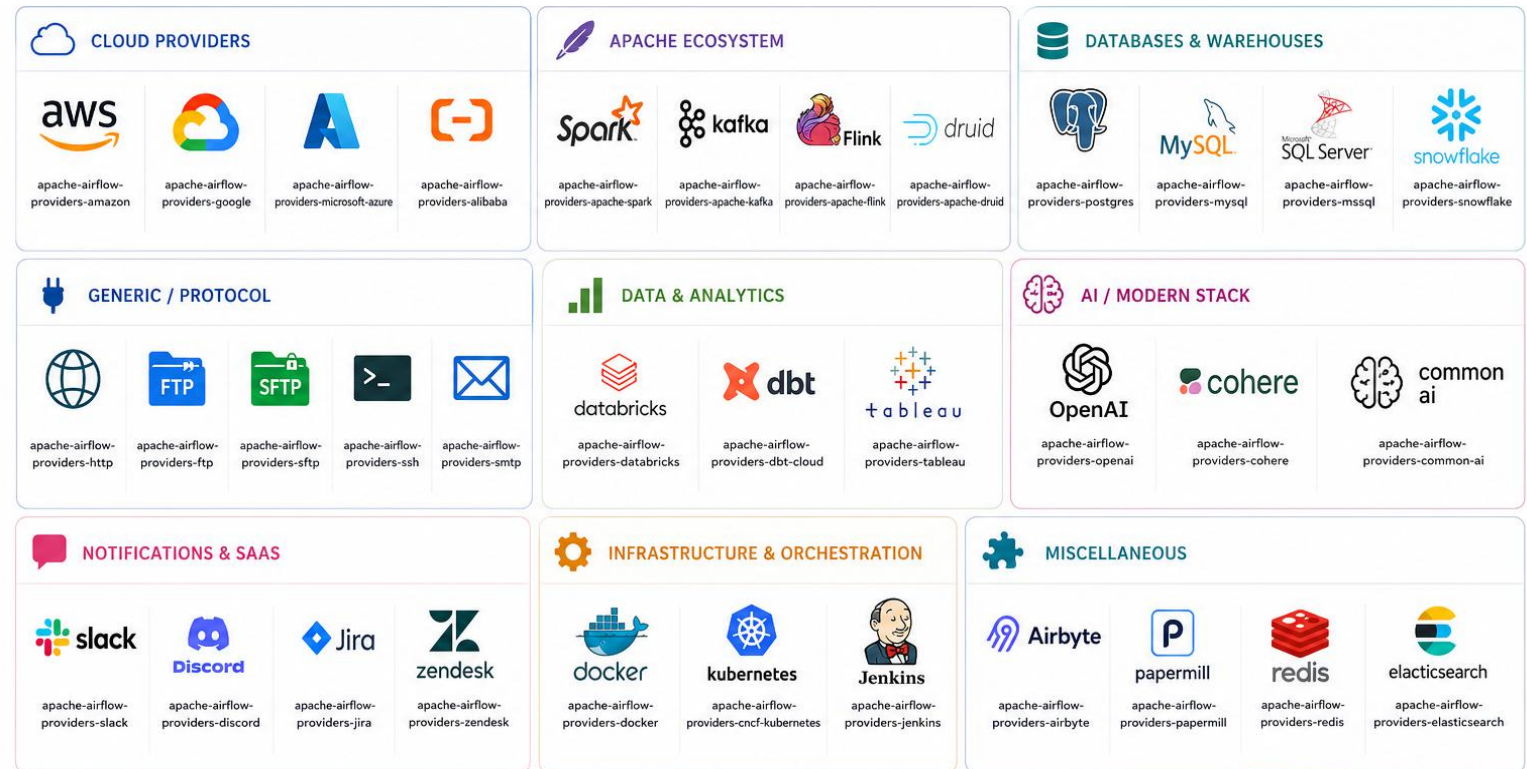


Провайдеры

Философия оркестраторов – **вынос** вычислений за пределы системы **менеджмента**.

Провайдеры – пакеты для взаимодействия с системами не из Airflow.

Более того, каждый может написать свой пакет-provider и опубликовать его.



Пользуемся провайдером

В учебных целях воспользуемся Apptainer-оператором.

```
pip install apache-airflow-providers-singularity
```

```
transform_with_apptainer = SingularityOperator(
    task_id="transform_with_apptainer",
    image="docker://python:3.11-slim",
    command="python3 /scripts/transform_ncbi.py",
    environment={
        "DATA_DIR": "/data",
        "GENE": "{{ params.gene }}",
        "DS": "{{ ds }}",
    },
    volumes=[
        f"{{DATA_DIR}}:/data",
        f"{{SCRIPTS_DIR}}:/scripts:ro",
    ],
)
```

Нужно указать
вытягиваемый образ,
команды, переменные
среды и примонченные
тома. Всё как при
создании контейнера
в **.def** файле **Apptainer**.

Запуск других DAGов

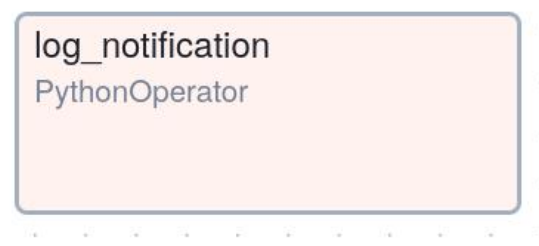
Не стоит различные по смыслу обработки объединять в единый DAG.

Решение: **TriggerDAGRunOperator**

Тут пока не
все вещи понятны!

```
trigger_notifier = TriggerDagRunOperator(  
    task_id="trigger_notifier",  
    trigger_dag_id="ncbi_pipeline_notify",  
    conf={"source_dag": "ncbi_gene_pipeline",  
        "gene": "{{ params.gene }}",  
        "execution_date": "{{ ds }}",  
        "report_path": "{{ ti.xcom_pull(task_ids='save_report',  
                                         key='report_path') }}" },  
    },  
    wait_for_completion=False,  
    trigger_rule=TriggerRule.ALL_DONE,  
)
```

Очень удобный
паттерн - как вы
думаете, почему?

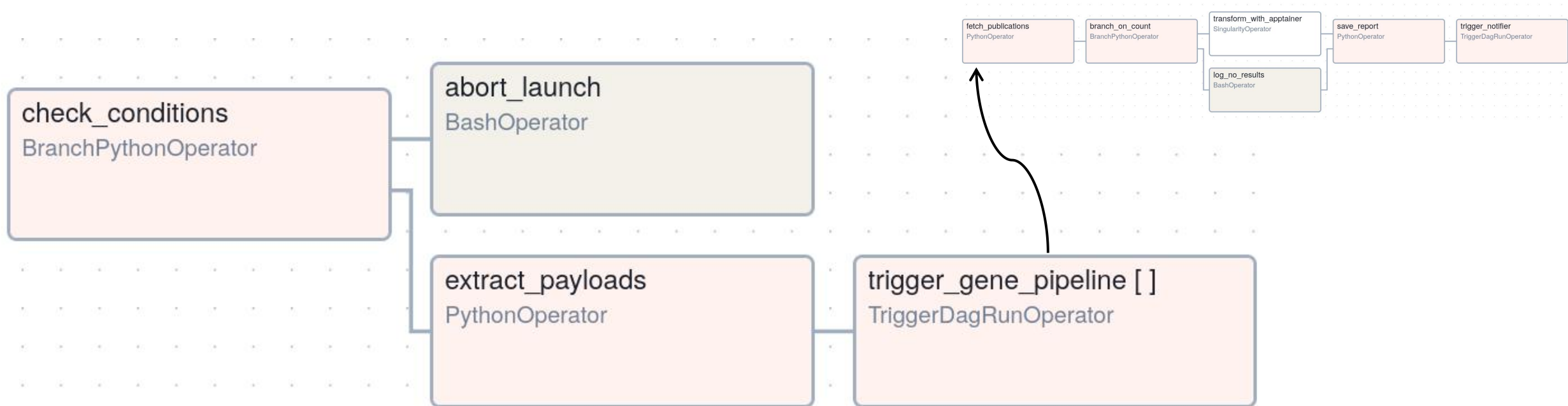


log_notification
PythonOperator

The diagram shows a light pink rectangular box with a thin grey border. Inside the box, the text 'log_notification' is on the top line and 'PythonOperator' is on the bottom line. Below the box, there are several small grey dots arranged horizontally.

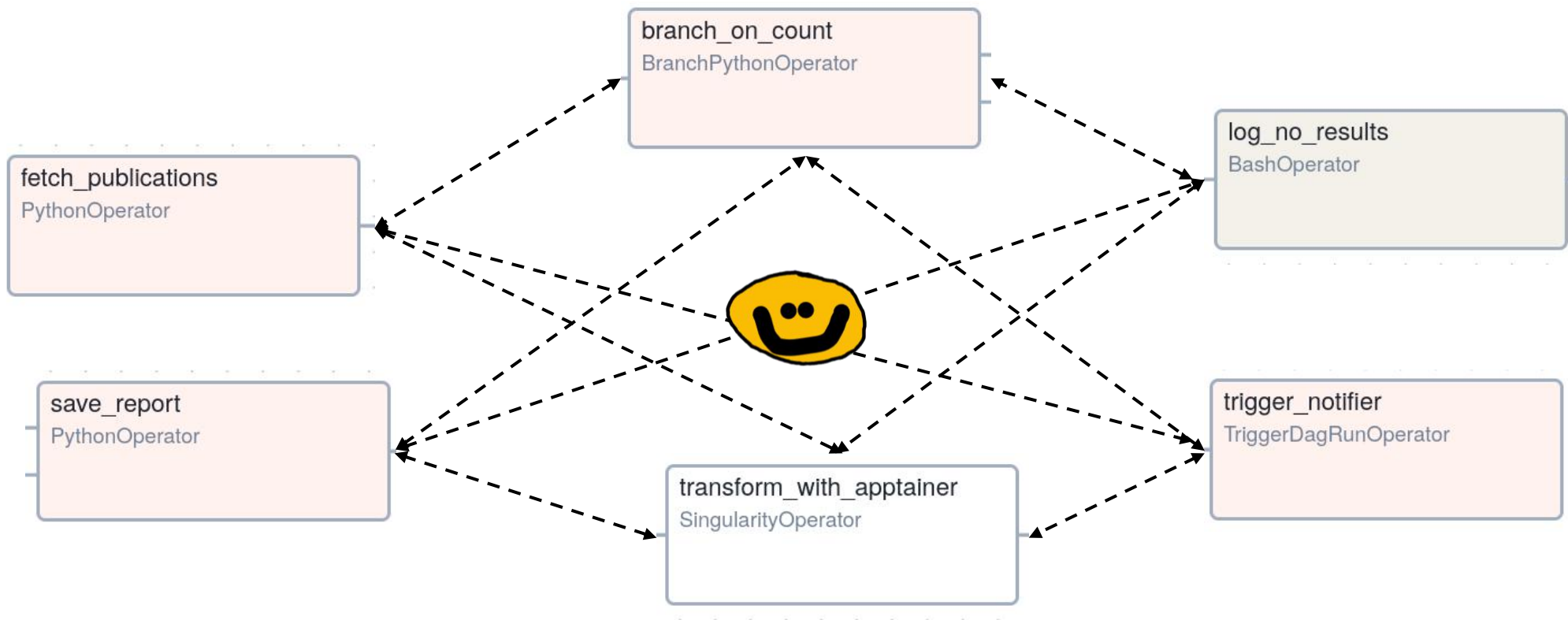
TriggerDAGRun

Удобство в том, что мы можем написать ДАГ для обработки набора данных на одних параметрах, и запускать их масштабно с помощью отдельного DAG для лонча (launch).



Собираем DAG!

1. Определим объект DAG.
2. Настроим зависимости между задачами.



Объект DAG

Можно использовать **как контекст**, можно – как декоратор (TaskflowAPI)

```
with DAG(  
    dag_id="pipeline_demo",  
    description="some DAG",  
    default_args=default_args,  
    start_date=datetime(2026, 1, 1),  
    catchup=False,  
    params={  
        "gene": "BRCA1",  
        "max_results": 20,  
    },  
) as dag:
```

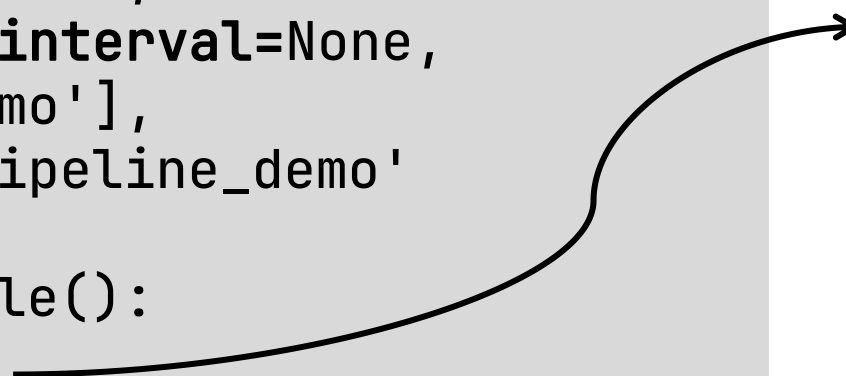


```
default_args = {  
    "owner": "airflow",  
    "depends_on_past": False,  
    "retries": 2,  
    "retry_delay": timedelta(minutes=5),  
}
```

Объект DAG

Можно использовать как контекст, можно –
как декоратор (TaskflowAPI)

```
@dag(  
    default_args=default_args,  
    description='some DAG',  
    start_date=datetime(2026, 1, 1),  
    catchup=False,  
    schedule_interval=None,  
    tags=['demo'],  
    dag_id='pipeline_demo'  
)  
def dag_example():  
    ...
```



TaskflowAPI удобнее.
Внутри него собираем задачи,
оборачивая их в **@task**.

```
@task  
def cleanup_temp():  
    temp_dir = '/tmp/dir'  
    if os.path.exists(temp_dir):  
        shutil.rmtree(temp_dir)  
        print(f"Removed {temp_dir}")
```

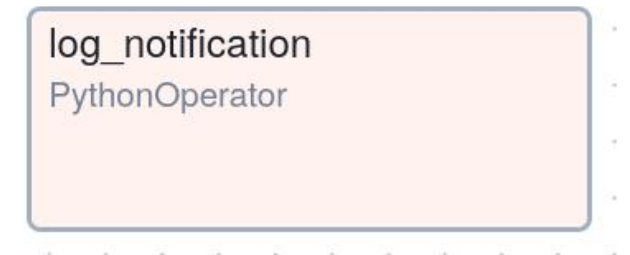
DAG целиком

```
with DAG(...) as dag:
    fetch_publications = PythonOperator(...)
    branch_on_count = BranchPythonOperator(...)
    transform_with_apptainer = SingularityOperator(...)
    log_no_results = BashOperator(...)
    save_report = PythonOperator(...)
    trigger_notifier = TriggerDagRunOperator(...)
    fetch_publications >> branch_on_count >> [transform_with_apptainer, log_no_results]
    transform_with_apptainer >> save_report
    log_no_results >> save_report
    save_report >> trigger_notifier
```

graph dependencies!

Остальные ДАГи:

- ncbi_pipeline_notify (вызов из основного)



- ncbi_launcher

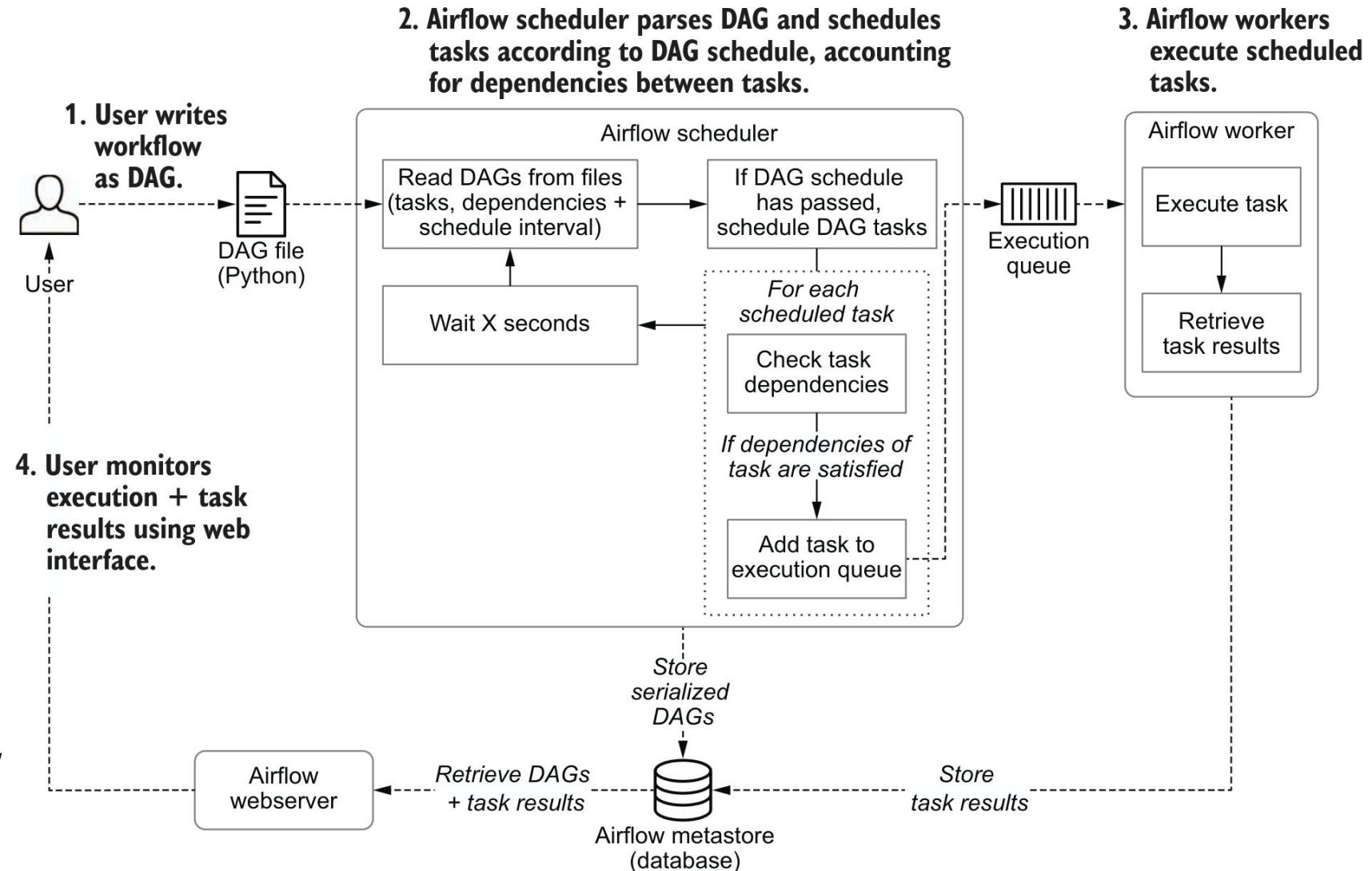


Жизненный цикл задач

Планировщик
это чуть более
сложный cron.
Умеет
проверять
зависимости.

`schedule_interval=None`

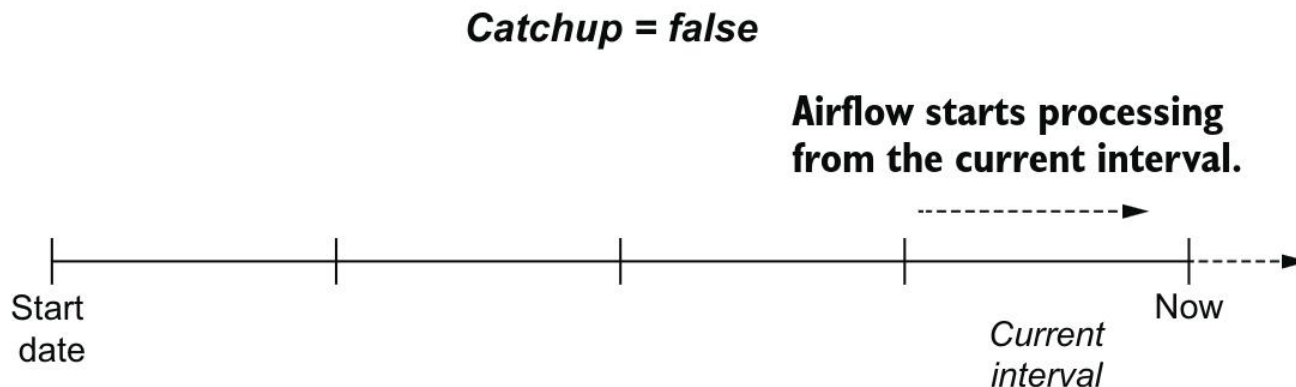
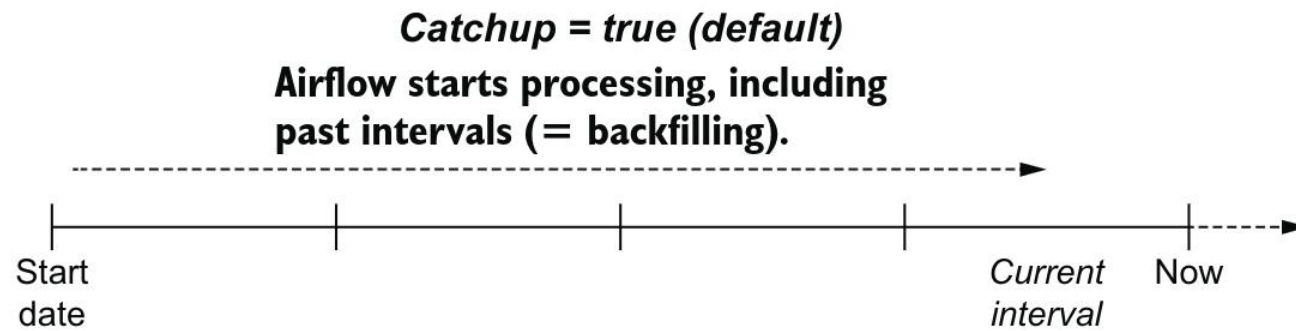
Поддержка выражений cron,
а также @weekly, @daily, ...



Backfill

Если DAG может быть
запланирован на будущее, то
его можно заставить и
восполнить прошлое

Preset	Meaning
@once	Schedule once and only once.
@hourly	Run once an hour at the beginning of the hour.
@daily	Run once a day at midnight.
@weekly	Run once a week at midnight on Sunday morning.
@monthly	Run once a month at midnight on the first day of the month.
@yearly	Run once a year at midnight on January 1.



Запуск DAG

Если хочется усложнить себе жизнь, у airflow есть CLI-интерфейс:

```
airflow dags trigger ncbi_pipeline_launcher \  
  --run-id manual_test_001 \  
  --conf '{  
    "genes": [  
      {"gene": "EZH2", "max_results": 50}  
    ]  
  }'
```

Этим обычно никто не пользуется, особенно в случаях интервальных запусков

Запуск DAG

DAGs

All 76

Active 3

Paused 73

Running 0

Failed 0

Filter DAGs by tag

Search DAGs

☐ Auto-refresh

DAG	Owner	Runs	Schedule	Last Run	Next Run	Recent Tasks	Actions	Links
ncbi_gene_pipeline bioinformatics demo ncbi	airflow	6	1 day, 0:00:00	2026-04-27, 15:38:13	2026-04-27, 00:00:00	5 1		
ncbi_pipeline_launcher2 launcher ncbi	airflow	1	None	2026-04-27, 15:37:16		7 1		
ncbi_pipeline_notify demo ncbi notify	airflow	6	None	2026-04-27, 15:39:24		1		

<<

<

1

>

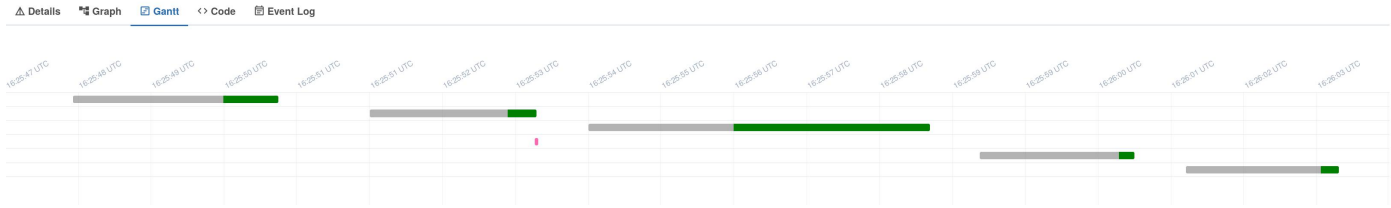
>>

Showing 1-3 of 3 DAGs

Включение DAG в список мониторинга интервалов

Кнопка включения (Trigger DAG Run)
Если указывали params в определении DAG, перекинет в новое окно.

Запуск DAG



Trigger DAG: [ncbi_gene_pipeline](#) Fetch NCBI PubMed data for a gene, transform, report, and notify

DAG Docs

dag_ncbi_pipeline.py

Demonstration Airflow pipeline:

1. Fetch gene publication IDs from NCBI PubMed (NCBI E-utilities, no auth needed)

2. Branch: proceed if results found, skip otherwise

3. Transform raw JSON inside an Apptainer container (SingularityOperator)

4. Save a human-readable report (PythonOperator)

5. Trigger a downstream notification DAG

Concepts demonstrated:

• DAG with params and Jinja templating ({{ ds }}, {{ params.gene }}, {{ ti.xcom_pull(...) }})

• PythonOperator, BashOperator, BranchPythonOperator, SingularityOperator, TriggerDagRunOperator

• XCom push/pull

• TriggerRule on a join task after branching

Select Recent Configurations

Default parameters

DAG conf Parameters

gene:

EZH2

max_results:

6

Generated Configuration JSON and Dagrun Options

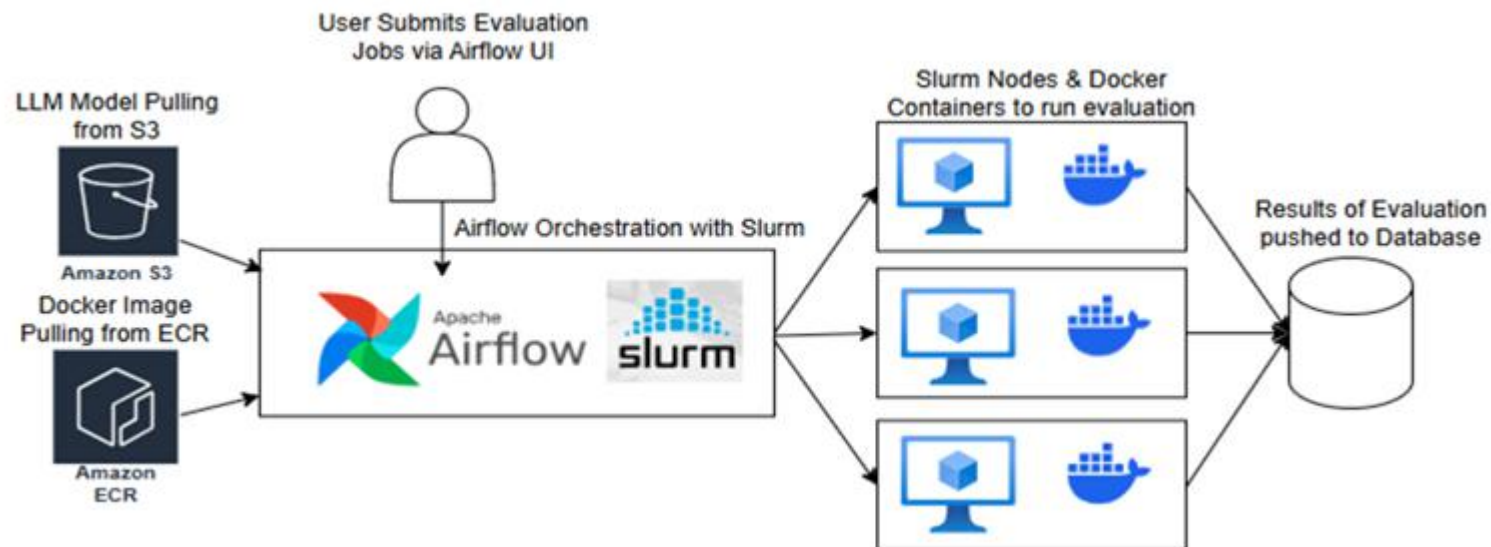
После запуска можно посмотреть график Gant выполнения задач, почитать логи, увидеть ошибки для отладки либо результаты.

```
{
  "gene": "EZH2",
  "execution_date": "2026-04-27",
  "total_hits": 268,
  "sampled_ids": [
    "42034032",
    "42029612",
    "42028972",
    "42024441",
    "42020715",
    "42020354"
  ],
  "id_count": 6
}
```

Ремарка: запуск на HPC

Airflow требует работы демонов для своих задач (scheduler). Такое не везде можно - админы часто блокируют такие процессы.

Склеить контейнер, чтобы задачи отдельных тасков отправлялись в SLURM - можно!



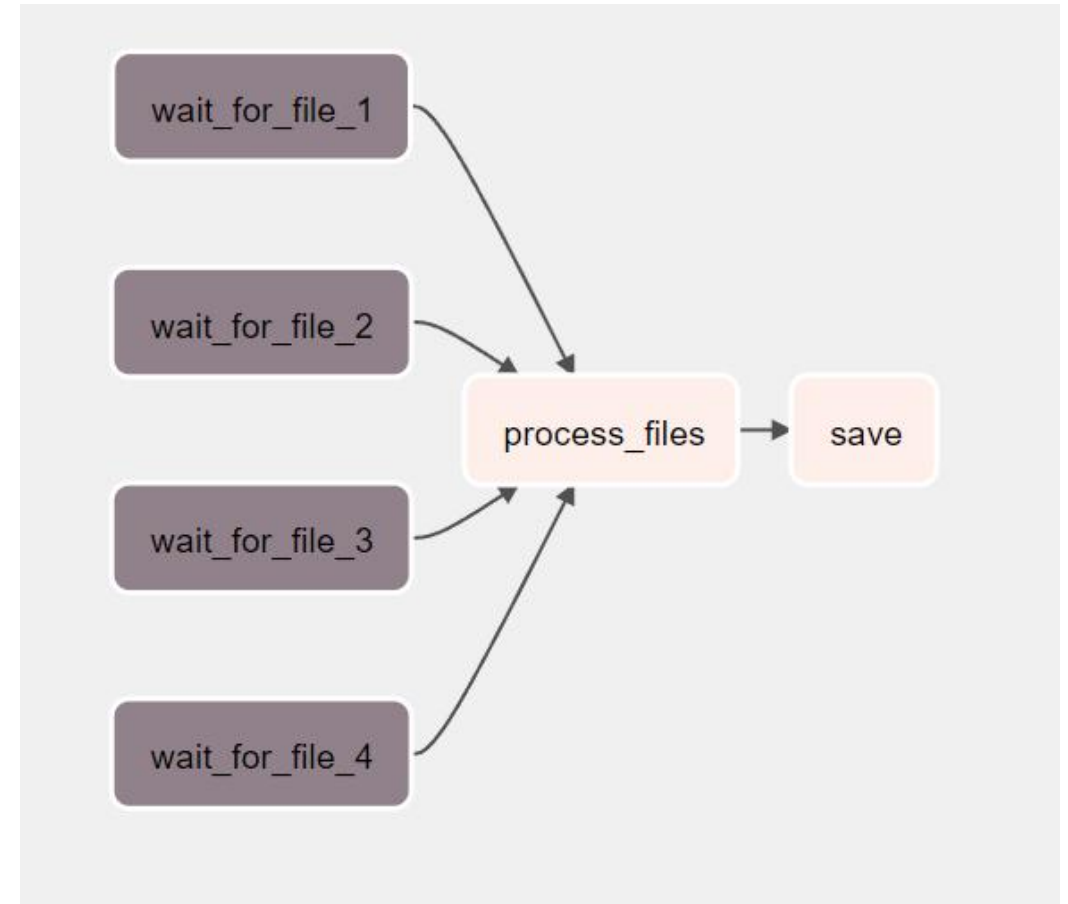
Airflow и SLURM

Как понять, что задача в SLURM отработала?

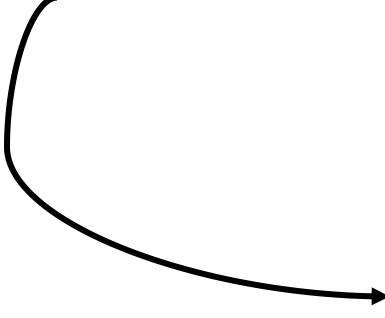
Другой тип операторов в airflow: сенсоры.

Сенсор - интервальный оператор, проверяющий условие.

Взаимодействие со SLURM'ом напишем в следующий раз. Контейнер для запуска airflow без рут-прав в листинге на Github



Другие типы операторов

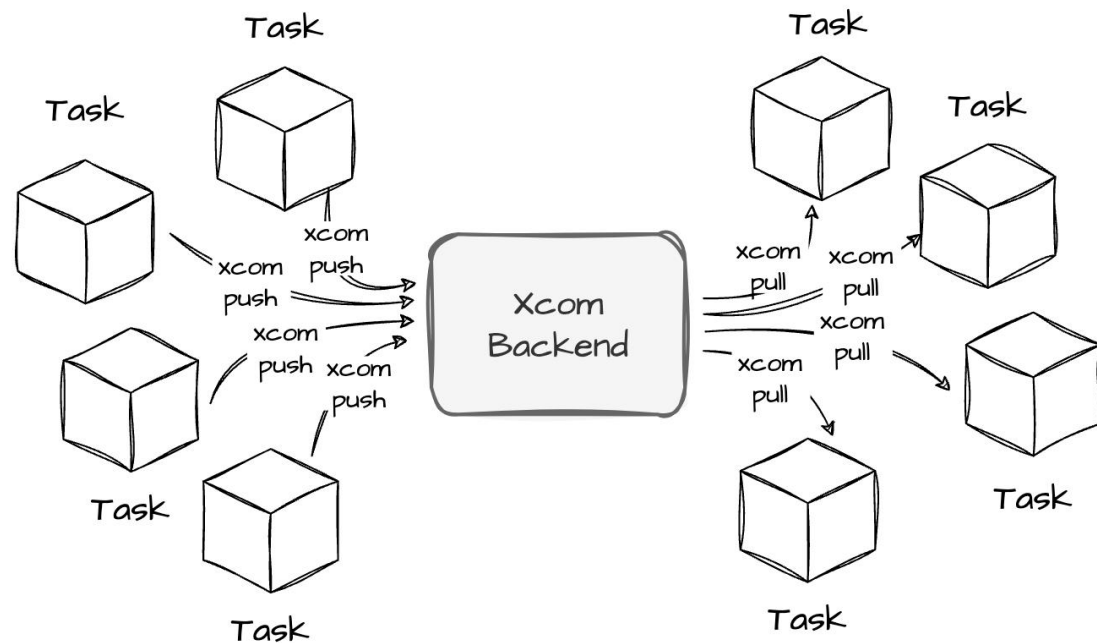
- Operator
- **Sensor**  Когда задача не может стартовать, пока не готовы входные данные например, **FileSensor**
- **Transfer**  **Для Load-задач:** перегон данных из хранилища (e.g., S3) в хранилище (e.g. MongoDB): например, **MongoToS3Operator**
- **Checker**  Проверка наличия и/или качества данных, чтобы мониторить такие ситуации. Например, **ValueCheckOperator**

Взаимодействие с мета-БД

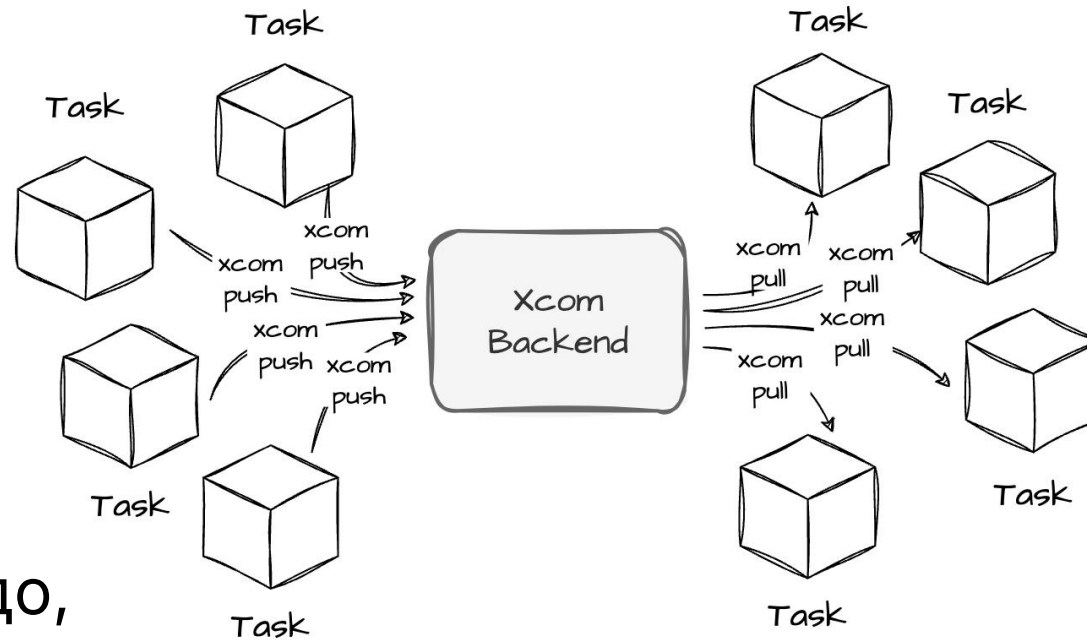
Каждый таск изолирован.

Что делать, если надо передать маленький кусок данных от одного к другому?

Пользуемся базой метаданных для кросс-коммуникации (**XCom**)



Взаимодействие с мета-БД



Таски, которым надо,
пушат в БД по ключу.

Другие таски (которым
тоже надо!) пуллят из БД
по ключу и task ID.

Идеально для метаданных (пути, конфиги). Размер ограничивается 2Kb, но можно и менять

XCom: напомним

- DAG `ncbi_pipeline`, task `save_report` пушит в БД:

```
def _save_report(gene: str, ds: str, **context) -> str:
    summary_path = os.path.join(DATA_DIR, f"summary_{gene}_{ds}.json")
    report_path = os.path.join(DATA_DIR, f"report_{gene}_{ds}.txt")
    with open(report_path, "w") as fh:
        ...
    context["ti"].xcom_push(key="report_path", value=report_path)
    print(f"[save_report] wrote {report_path}")
    return report_path
```

Переменная `context` всегда передаётся каждому taskу, это его контекст выполнения (видели раньше). В БД по `ti` (task ID) пушится путь `report_path`.

XCom: напомним

- DAG `ncbi_pipeline`, task `trigger_notifier` обращается в БД:

```
trigger_notifier = TriggerDagRunOperator(
    task_id="trigger_notifier",
    trigger_dag_id="ncbi_pipeline_notify",
    conf={...,
        "report_path": "{{ ti.xcom_pull(task_ids='save_report', key='report_path') }}",
    },
    wait_for_completion=False,
    trigger_rule=TriggerRule.ALL_DONE,
)
```

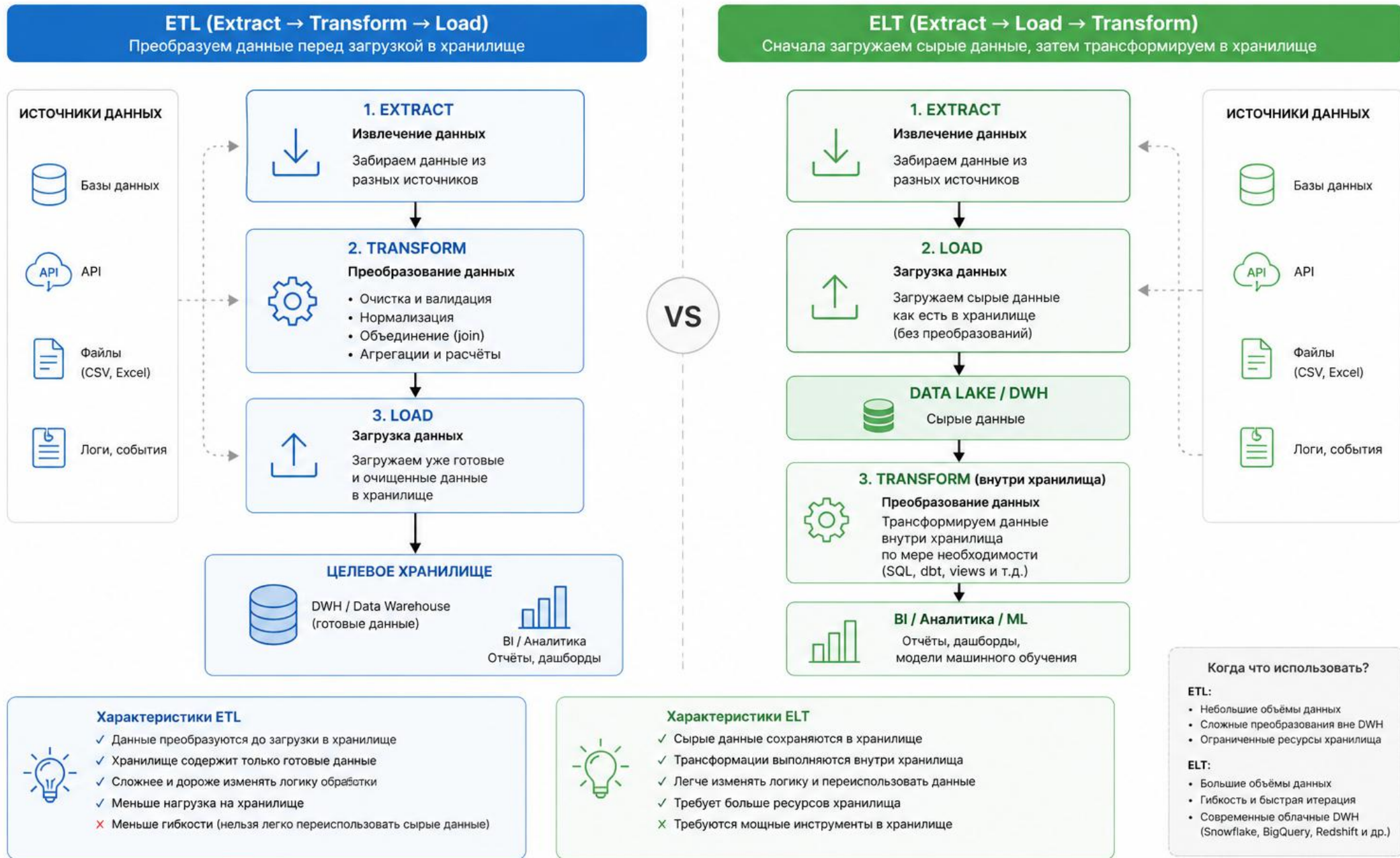
Здесь `context` шаблонизирован и внутри шаблонизации происходит обращение в БД по `ti` (task ID) и ключу `report_path`.

В следующий раз

- Сенсоры (подробнее)
- Хуки
- Соединения (Connections)
- Переменные (Variables)
- Секреты
- Управление воркерами
- ...практика?

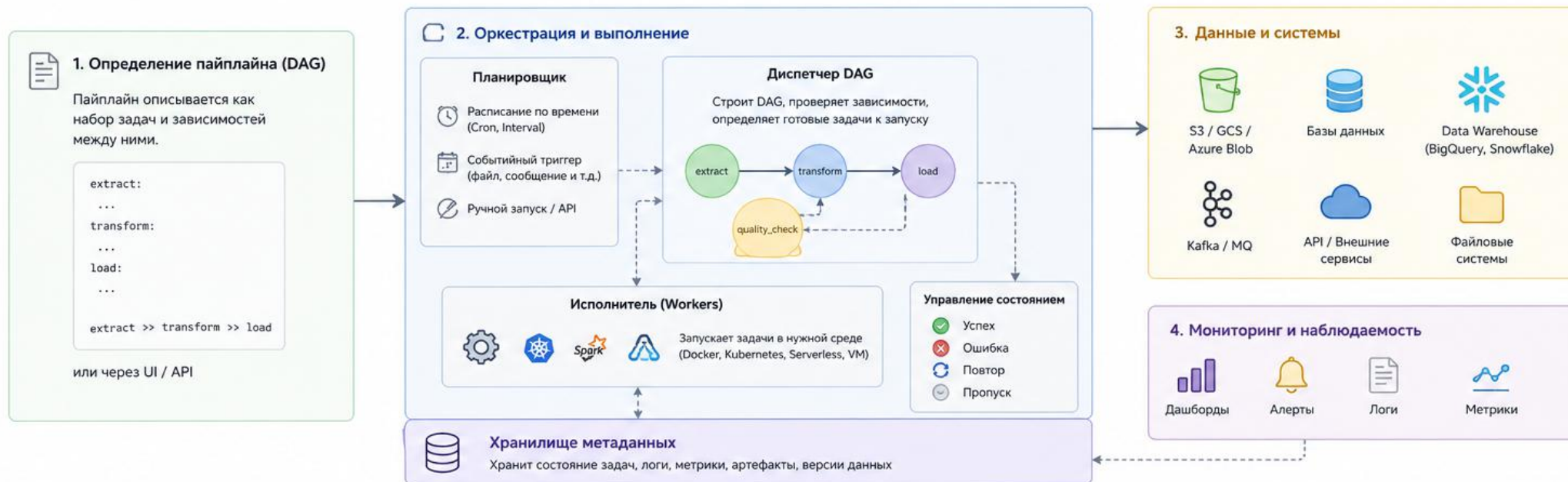
ETL vs ELT

Два подхода к обработке данных

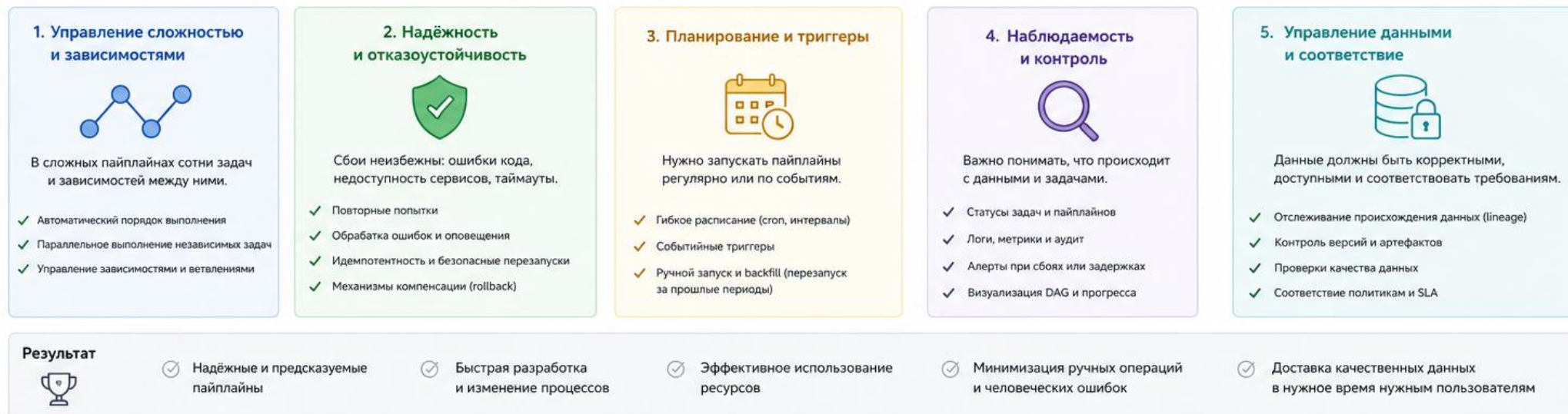


Как работают современные системы оркестрации пайплайнов обработки данных

Оркестратор управляет выполнением пайплайнов – определяет порядок задач, контролирует их выполнение, обрабатывает зависимости, ошибки и повторные попытки.



Какие проблемы и задачи решают системы оркестрации



Apache Airflow – Metadata Database (ERD)

Основные таблицы и связи

